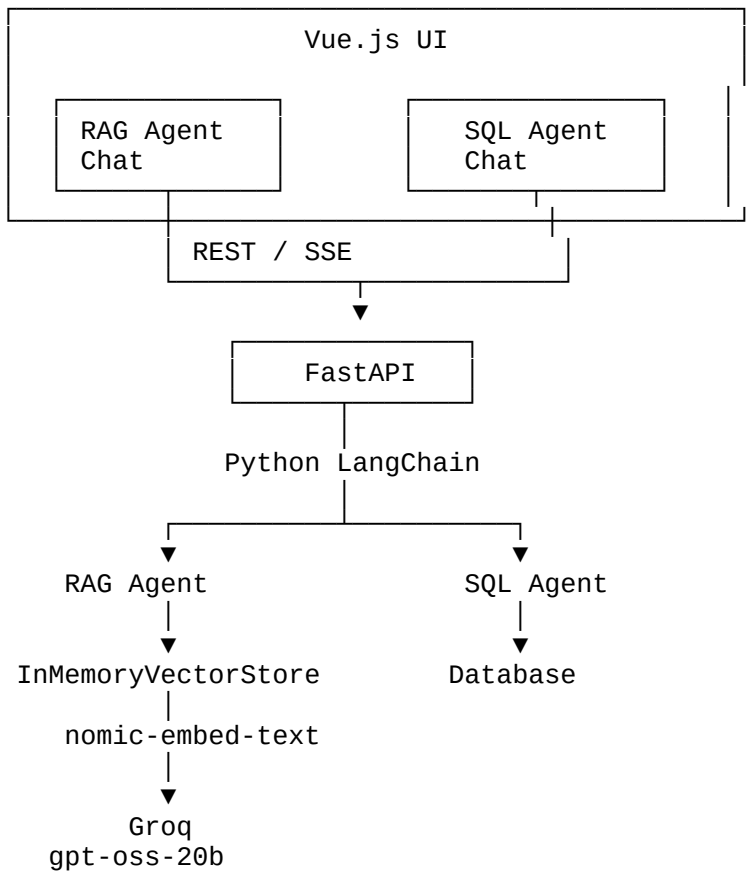


Yes — that is a **very good demo stack**, and I would keep it almost exactly as you have it.

The important thing is to keep the architecture simple:



Your stack

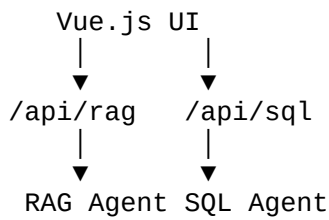
Component	Your choice	My view
Runtime	Node.js 22.22.1	Fine for Vue tooling
UI	Vue.js	Good
API	FastAPI	Excellent
Agent	Python LangChain	Good
LLM	Groq openai/gpt-oss-20b	Good for demo
Vector store	InMemoryVectorStore	Perfect for demo
Embeddings	nomic-embed-text	Good
SQL Agent	LangChain SQL tools	Good

One important architectural decision

I would **not** make one giant agent that decides between RAG and SQL initially.

Build two clearly separated agents:



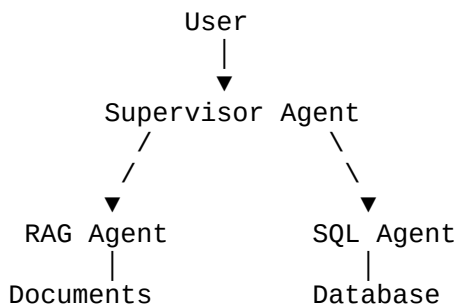


This makes your demo much easier to explain.

For example, your UI could have:

AI Business Assistant	
[Documents]	[Database]
Ask questions about company documents	Ask questions about business data

Then later you can combine them into a **supervisor/router agent**:



That becomes a much more interesting **Digital Employee** demo.

I would make only one change

Your:

InMemoryVectorStore

is excellent for the demo **if the application is single-instance and you don't care about persistence**.

But understand the limitation:

```

Upload documents
  ↓
InMemoryVectorStore
  ↓
Works
  ↓
Restart FastAPI
  
```

↓
喀 Documents/embeddings disappear

For a customer demo, that's completely acceptable.

You can preload your demonstration documents when FastAPI starts:

```
FastAPI startup
↓
Load demo PDFs
↓
Chunk
↓
nomic-embed-text
↓
InMemoryVectorStore
↓
Ready
```

Then the customer opens the website and everything is already there.

That is actually **simpler than building document persistence right now**.

Your RAG Agent

I'd make the RAG agent responsible for:

```
Question
↓
Retrieve documents
↓
Select relevant chunks
↓
Groq
↓
Answer
↓
Sources
```

The response should be structured rather than just returning a string:

```
{
  "answer": "The policy covers windshield damage subject to the deductible.",
  "sources": [
    {
      "document": "auto_policy.pdf",
      "page": 12,
      "content": "Windshield and glass damage..."
    }
  ]
}
```

That `sources` field is important for your customer demo.

Your SQL Agent

For SQL, I'd expose something like:

POST /api/sql/chat

Request:

```
{
  "message": "What was revenue by legal entity in 2025?"
}
```

The agent:

User question



SQL Agent



Understand schema



Generate SQL



Execute SQL



Analyze result



Natural-language answer

Response:

```
{
  "answer": "Total recognized revenue in 2025 was $12.4M.",
  "sql": "SELECT ...",
  "rows": [
    ...
  ]
}
```

For a technical customer, showing the generated SQL is **very impressive**.

You can have:

Answer

\$12.4M recognized revenue

▼ Show SQL

```
SELECT legal_entity,
       SUM(revenue)
FROM revenue
WHERE fiscal_year = 2025
GROUP BY legal_entity;
```

Your API should stay tiny

I wouldn't create 20 endpoints.

Start with:

```
POST /api/rag/chat
POST /api/sql/chat
```

```
POST /api/rag/chat/stream
POST /api/sql/chat/stream
```

```
GET /api/health
```

Maybe later:

```
POST /api/documents/upload
```

That's enough.

One thing I'd strongly recommend

Use **SSE** rather than normal REST responses for the chat.

Your Vue UI can show:

User:
What does the policy cover?

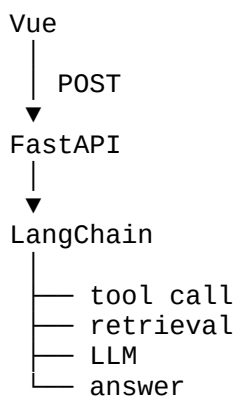
AI:
Let me search the relevant documents...

AI:
The policy covers...

Sources:
憲 Auto Policy – Page 12

instead of waiting for the entire agent execution.

Architecture:



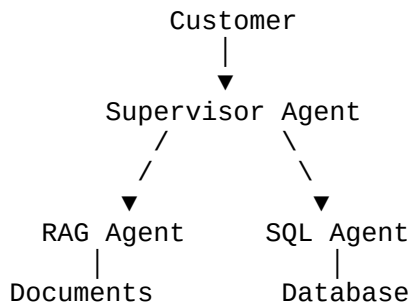
↓ SSE
Vue

The final demo I would build

Your website could have three tabs:

crabAI
[RAG Agent] [SQL Agent] [Combined]
Conversation Customer: What does the policy cover? AI: The policy covers... Sources: Auto Policy.pdf – Page 12

Then **Combined** later:



Example:

"Based on our claims policy, how many claims last year were denied because of excluded windshield damage?"

The supervisor could:

RAG Agent
↓
Find policy exclusion
↓
SQL Agent
↓
Query claims database
↓
Supervisor

↓
Final answer + sources + SQL